

Proyecto 2

Taller de Aprendizaje Automático

Valentina Alaniz, Estéfano Bargas, Betina Cerecetto

Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay

Julio 2024

Índice

1. Introducción	2
1.1. Métrica de evaluación	2
2. Exploración y visualización de datos	2
3. Gestión de Datos	3
4. Modelado	3
4.1. Red CNNs	3
4.2. Transfer learning con MobileNet	3
4.2.1. Modelo inicial	3
4.2.2. Modelo regularizado	5
4.2.3. Duración de los audios y base temporal	5
4.3. Modelo Custom Made	6
5. Mejoras y Optimización	6
5.1. Aumento de datos	6
5.1.1. Datos Ruidosos	6
5.1.2. Transformaciones en audios	6
5.1.3. SpecAugment	6
5.1.4. MixUp	7
6. Análisis de resultados	8
6.1. Registro de experimentos	8
6.2. Resultados de Kaggle	8
7. Conclusiones	8
7.1. Limitaciones y Trabajo Futuro	8

1. Introducción

El objetivo de este proyecto es crear un modelo el cual permita a partir de grabaciones de sonidos lograr etiquetarlos en 80 categorías distintas, cada audio puede pertenecer a más de una categoría. Con este fin se utilizaron los datos para entrenar los modelos a partir de una competencia ya existente de *Kaggle* llamada [Freesound Audio Tagging 2019](#). El conjunto de entrenamiento utilizado es el de la competencia proporcionada por el curso [TAA 2024 - Freesound Audio Tagging](#).

Resolver este problema es importante debido a que evita realizar la tarea de forma manual, que actualmente se lleva a cabo de esa manera. Además, aunque algunos sonidos son fácilmente identificables por su origen, hay otros como el sonido de un extractor o un compresor que son difíciles de distinguir para una persona.

1.1. Métrica de evaluación

La métrica de evaluación utilizada se denomina *label-weighted label-ranking average precision* (LWLRAP), una variante de *label-ranking average precision* (LRAP). *LRAP* se utiliza para calcular un puntaje global sobre todas las muestras y etiquetas, con la característica de que a cada muestra se le asigna el mismo peso (es decir, si una muestra tiene predichas mas etiquetas que otra, entonces se penaliza, logrando que las muestras con menos etiquetas no sean irrelevantes). Por otro lado, *LWLRAP* asigna mismo peso a cada etiqueta individualmente, lo que implica que las muestras con más etiquetas tienen mayor influencia en el puntaje final. Esta característica de *LWLRAP* facilita la obtención de un puntaje específico para cada etiqueta.

Entonces a la hora de utilizar la métrica debemos tener en cuenta la proporción de cada label en el conjunto de validación y debemos aprovechar el puntaje para cada label. Idealmente un buen conjunto de validación representa cada label en igual proporción que el conjunto de test, sin embargo como no conocemos la distribución de test y como la métrica otorga mismo peso a cada label, decidimos que lo mejor es tener una distribución lo más uniforme posible en validación. Este es el comportamiento por defecto de un *train-test split*.

Se modificó la implementación de la métrica brindada por los organizadores del challenge para obtener un acumulador de score utilizando operaciones de *TensorFlow* para convertirlo en una métrica compatible con modelos de *Keras*. Esto llevo bastante prueba y error.

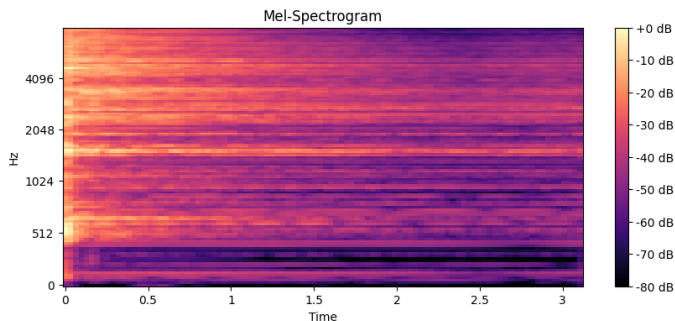
2. Exploración y visualización de datos

Inicialmente, se exploraron los audios en el formato especificado (.wav), donde se observaron diferencias entre los datos como la duración de cada audio y diversidad del contenido. El modelo tiene que ser capaz de distinguir cada clase de sonido de las 80 definidas (conversaciones, ruidos mecánicos, sonidos de la naturaleza, etc).

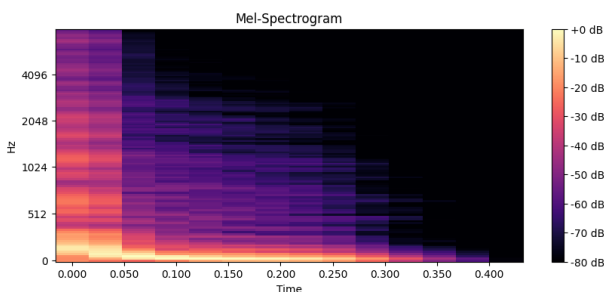
Posteriormente, los audios fueron transformados en espectogramas. Los espectogramas son representaciones visuales bidimensionales de la transformada de Fourier de un audio, donde el eje horizontal representa el tiempo, el eje vertical representa las frecuencias, y los colores indica la intensidad de cada frecuencia en un momento dado.

Para adaptar la visualización al modo en el que el oído humano percibe los sonidos se empleó la [escala logarítmica Mel](#). El oído humano no percibe las frecuencias de manera lineal, somos más sensibles a las vibraciones en frecuencias bajas que en frecuencias altas. Esta escala refleja esta percepción logarítmica dando más peso a las frecuencias bajas y menos a las altas.

En la [Figura 1a](#) correspondiente al instrumento charleston se observa alta intensidad en la banda de frecuencias mas altas, con una duración de 3 segundos. En cambio en la [Figura 1b](#) correspondiente a un tambor se destaca la intensidad de las frecuencias mas bajas durante un periodo mucho mas corto de tiempo.



(a) Espectrograma Log-Mel para muestra de categoría Hi-Hat



(b) Espectrograma Log-Mel para muestra de categoría Bass Drum

Para el entrenamiento de las redes neuronales empleadas para la resolución de la tarea, se utilizarán espectrogramas log-mel.

3. Gestión de Datos

Para este proyecto, debido al gran volumen de datos con el que se trabaja, fue necesario gestionarlos de manera eficiente. Por ello, se optó por utilizar la estructura de *Dataset* mediante la API de tensorflow, en lugar de almacenar los datos en arreglos de *numpy*, lo cual resulta poco escalable para la magnitud de datos manejados. La *API* nos permite implementar técnicas como el *Prefetch*, que prepara un batch de datos en paralelo mientras otro *batch* está siendo procesado en entrenamiento, optimizando el uso de la memoria RAM. La cantidad de batches que se preprocesan es elegida automáticamente por TensorFlow.

Los algoritmos de procesamiento de datos se diseñaron de manera concurrente para aprovechar todos los hilos disponibles en las máquinas utilizadas. Se organizaron los conjuntos de validación y entrenamiento en diferentes carpetas, al igual que sus versiones aumentadas utilizando técnicas explicas en futuras secciones.

4. Modelado

4.1. Red CNNs

Como se mencionó en el Proyecto 1, la capacidad de computo es usualmente una limitante en problemas relacionados con el aprendizaje automático. En el proyecto anterior, se utilizaron redes neuronales **fully connected**, donde estas limitaciones fueron evidentes. Además, estas redes tienen una gran cantidad de parámetros debido a que cada neurona de una capa, incluida la del sesgo, se conecta con todas las neuronas de la siguiente capa, lo que resulta en un modelo poco escalable y propenso al sobre-ajuste. Cada neurona procesa la totalidad de los datos, y cuando se trabaja con imágenes, muchas veces no es beneficioso.

Por ejemplo, una imagen pequeña de 16x16 píxeles contiene 256 píxeles, por cada píxel existe una neurona en la capa de entrada, donde cada una se conecta con cada neurona de la siguiente capa, así sucesivamente. Esto resulta en un gran número de parámetros a ajustar, lo que aumenta el costo computacional. La cantidad de neuronas aumenta significativamente con la cantidad de píxeles de la imagen a procesar.

En este proyecto se utilizarán redes neuronales convolucionales, las cuales se caracterizan por su capacidad para procesar imágenes de manera eficiente. La convolución es la operación central de este tipo de redes, implica el desplazamiento de un filtro (también llamado *kernel*) sobre la imagen para extraer las características esenciales, permitiendo detectar patrones como bordes, texturas y otros detalles relevantes.

Estos filtros generalmente son más pequeños que las imágenes completas, por lo que al realizar la convolución resulta en salidas reducidas para las siguientes capas. A diferencia de las redes **fully connected**, donde cada neurona observa toda la imagen, en las convolucionales, cada neurona se centra en una región local. Esto es beneficioso porque permite identificar características locales específicas, sin ser computacionalmente muy costoso. Además, muchas veces la información que necesita aprender la red para resolver un problema de este tipo se identifica mediante características locales.

Después de una capa de convolución, suele seguir una capa de *pooling* que reduce la dimensión de los datos mediante submuestreo. Este proceso disminuye el costo computacional y aunque implica una pérdida de información, ayuda a condensar características esenciales aprendidas en la capa anterior. La pérdida de detalles muy finos puede ser beneficiosa para enfocar el modelo en rasgos más generalizables.

Como se discutió en la [Sección 2](#), los diferentes sonidos presentan patrones particulares en los espectrogramas. Por esta razón, las redes neuronales convolucionales (CNN's) son adecuadas para aprender estas características visuales específicas (dado un correcto preprocesamiento de los audios para destacar la firma auditiva de cada clase). Utilizan filtros para procesar localmente áreas específicas de la imagen, permitiendo que la red identifique sonidos incluso si hay variaciones temporales en el audio. Por ejemplo, si una red aprendió a reconocer el espectrograma bass-drumm de [Figura 1b](#) podría identificar correctamente el sonido incluso si el instrumento se toca medio segundo más tarde de lo que lo hizo en el entrenamiento.

4.2. Transfer learning con MobileNet

4.2.1. Modelo inicial

El transfer learning minimiza tanto el tiempo de entrenamiento como los requerimientos de memoria, aprovechando el rendimiento de un modelo complejo pre-entrenado. Es importante esta característica para resolver el problema, ya que no contamos con los recursos computacionales o la cantidad de datos suficientes como para entrenar desde cero una red neuronal convolucional profunda.

La arquitectura evaluada utiliza la técnica de transfer learning basándose en *MobileNetV2*, un modelo de red neuronal convolucional desarrollado para dispositivos móviles, por lo que es relativamente ligera y eficiente. La versión 2 de este modelo tiene menor cantidad de parámetros. Al modelo base se le suma una capa *GlobalAveragePooling2D* seguida por una capa densa final con 80 neuronas (una por cada etiqueta), utilizando activación *sigmoide* para permitir la clasificación multi-etiqueta.

Además, se ajustan los canales de entrada de las imágenes para normalizar los valores al rango $[-1; 1]$. En la Figura 2 se muestra un diagrama de la arquitectura del modelo. Este modelo será usado para la mayor parte de la experimentación.

Para el entrenamiento, se congela el modelo base y solo se entrenan las capas agregadas durante 7 épocas, utilizando el optimizador *Nadam* con su learning rate por defecto y la función de pérdida de entropía cruzada binaria. Esto simula tener 80 predictores binarios, uno para cada clase. Posteriormente, se realiza fine-tuning descongelando los últimos dos bloques del modelo (decisión tomada arbitrariamente) y se re-entrena con un learning rate reducido por dos órdenes de magnitud.

Siguiendo el baseline indicado por los docentes, se entrena utilizando los espectrogramas *log-mel* del conjunto *curated*, generados con la librería *librosa* y aplicando enventanado de Hann (que prioriza la distinción entre señales de frecuencias cercanas). Se utiliza la duración completa de cada audio (o sea cada espectrograma tiene distinta base temporal). Se reserva un 20 % de los datos para validación de manera estratificada. El tamaño de las imágenes utilizado es el más grande aceptado por el modelo base, 224x224 píxeles. Se prueban tanto los audios con su frecuencia de muestreo original (44.1 kHz) como los re-muestreados a 16 kHz.

En la Figura 3, se grafican los valores de *lwlrwrap* para los conjuntos de entrenamiento y validación tanto para la fase de entrenamiento inicial como para el fine-tuning del modelo, en el caso de utilizar los audios con muestreo original y los audios remuestreados. De aquí se desprenden algunas conclusiones:

- La medida mas alta de *LWLRAP* en validación fue de 0.62, lo cual es un buen punto de arranque, demostrando el potencial del *transfer learning*.
- El modelo entrenado con los audios re-muestreados muestra una leve pérdida de rendimiento (diferencia de 0.02), a cambio de una reducción del 50 % en el tamaño de los datos. A partir de ahora, se elige utilizar los audios re-muestreados ya que no perdemos casi rendimiento y facilita el manejo de datos, así como la capacidad de realizar mayor aumento de datos en futuros experimentos.
- Contrario a lo esperado, la fase de *fine-tuning* comienza con un rendimiento inferior al de la fase anterior. Se atribuye a que la primera época produce cambios en los parámetros de las capas desbloqueadas, resultando en una salida diferente del modelo base, y la capa densa final no logra adaptarse a los cambios a tiempo.
- En la fase de *fine-tuning* el modelo presenta un extremo sobreajuste y, después de 3 épocas, no mejora el rendimiento en validación. El entrenamiento se detiene debido al callback del *early stopping*. Esto puede deberse a la falta de datos suficientes como para entrenar capas más complejas del modelo base.

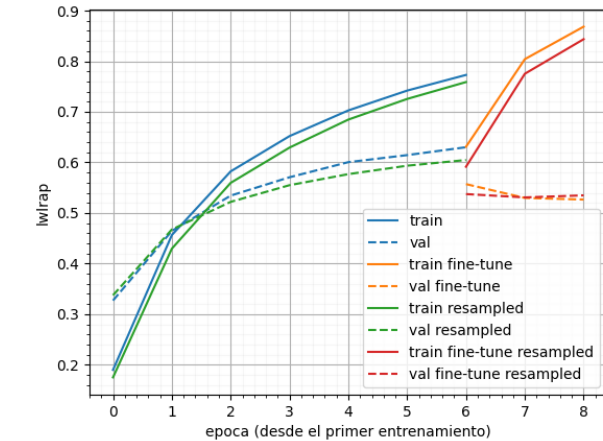


Figura 3: *LWLRAP* de primeros entrenamientos de transfer learning con base *MobileNetV2*.

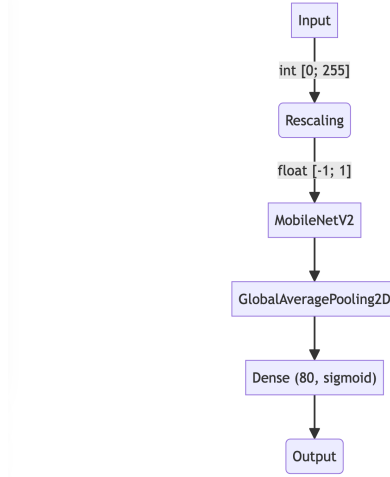


Figura 2: Diagrama de la arquitectura del modelo de transfer learning basado en *MobileNetV2*.

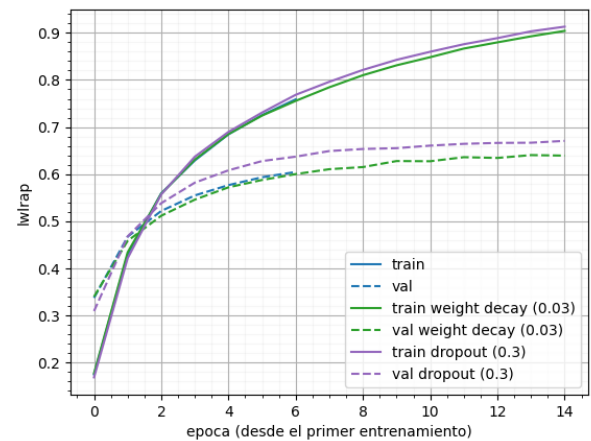


Figura 4: *LwLrap* de primeros entrenamientos de transfer learning con base *MobileNetV2*.

4.2.2. Modelo regularizado

Para disminuir el sobreajuste, se probaron dos métodos de regularización por separado: modificar la tasa de *Dropout* del modelo base y aplicar weight decay con *AdamW*. Para estos experimentos, se decidió aumentar la cantidad de épocas máximas a 15. Para hallar los mejores hiperparámetros de regularización, se realizó una búsqueda utilizando *Hyperband* de *KerasTuner*, asignando diferentes espacios de búsqueda para cada técnica (0.001 a 0.5 para *Dropout* y 0.001 a 1 para weight decay). Los resultados se observan en la Figura 4. Se omiten los datos de las fases de fine-tune ya que finalizan con resultados inferiores a los del modelo no regularizado, complicando la visualización de los datos relevantes.

Los valores óptimos obtenidos fueron: 0.03 para weight decay y 0.3 la tasa de *Dropout*. Estos resultados fueron añadidos como referencia al modelo sin regularización. Se destaca que, aunque los modelos regularizados mantienen una tendencia similar durante el entrenamiento, en validación el modelo con *Dropout* mejoró su rendimiento (*LWLRAP* final en validación de 0.60 comparado con 0.67 del modelo sin regularizar). El valor obtenido para weight decay no fue lo suficientemente alto como para tener un impacto significativo, lo que sugiere que la búsqueda no encontró un parámetro de weight decay que mejore el rendimiento del modelo en validación.

Dado este resultado, se decidió utilizar este modelo con una regularización *Dropout* al 0.3, utilizando los audios re-muestreados para la siguiente serie de experimentos relacionados a los datos de entrenamiento. Aunque inicialmente la fase de *fine-tuning* no mejora el modelo, se continuará explorando esta técnica para determinar si aumentando la cantidad de datos, se logra entrenar eficazmente las últimas capas del modelo base.

4.2.3. Duración de los audios y base temporal

La duración de los audios es un parámetro importante que afecta la escala temporal de los espectrogramas. Los audios más largos producen espectrogramas más densos, mientras que los más cortos resultan en espectrogramas visualmente pixelados. Un enfoque para explorar es fijar la duración de los audios a un valor y mantener las escalas de tiempo constantes. Dado que la gran mayoría de los sonidos en los datos de entrenamiento son de muy corta duración (menos de dos segundos), y considerando que los sonidos mas largos suelen ser cíclicos, se optó por un estándar de 5 segundos. Este tiempo es suficiente como para capturar la duración completa de la mitad de los audios en el conjunto de entrenamiento.

Se realizaron dos experimentos: primero, todos los audios (tanto los de entrenamiento como los de validación) fueron recortados a los primeros 5 segundos (experimento denominado *clip*), y segundo, se recortaron a los primeros 5 segundos los audios en validación, mientras que en entrenamiento cada audio fue dividido en segmentos de 5 segundos o menos (denominado *split*). Esta última estrategia aumentó el total de muestras a 8320 en el conjunto de entrenamiento. En la Figura 5 se observan los resultados de ambos experimentos en contraste a entrenar con largo variable. En los tres casos el modelo llega al mismo rendimiento final en validación, sin embargo se nota el leve efecto regularizador que tiene fragmentar los audios. Dado que la gran cantidad de audios complica el manejo de archivos para otras técnicas de aumentación (bajo las restricciones de nuestras máquinas) y la posibilidad de perder información importante al cortar los audios del conjunto de validación (y por lo tanto también los de test) se decide no aplicar esta técnica en experimentos futuros. El resultado del experimento en la fase de fine-tuning no arroja resultados relevantes, por lo cual se omite su presentación en este informe.

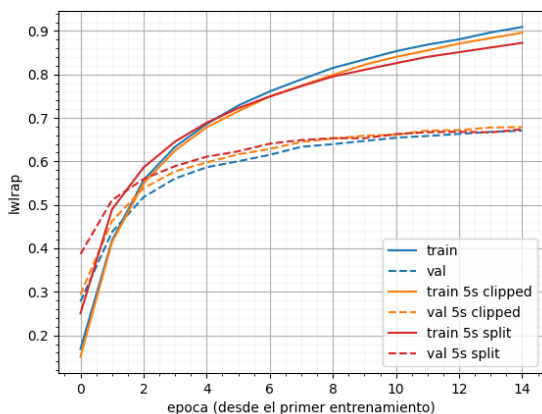


Figura 5: LwLrap de entrenamiento para modelo MobileNet regularizado, en comparación al mismo modelo con audios cortados a los primeros 5s (*clip*) y audios partidos en 5s (*split*).

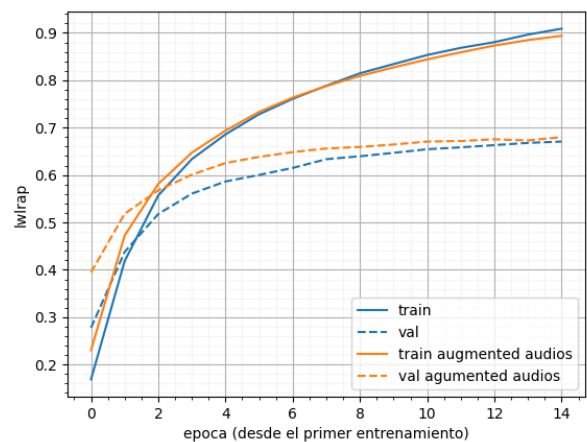
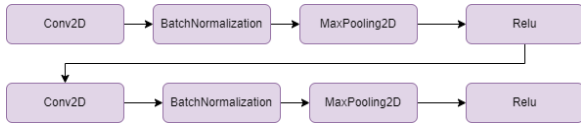


Figura 6: LwLrap de entrenamiento para modelo basado en MobileNet regularizado, comparado al utilizar aumento en base a transformaciones de audios.

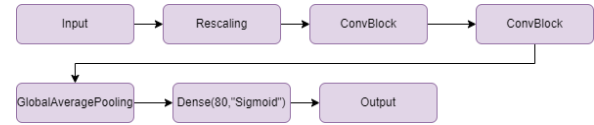
4.3. Modelo Custom Made

Además de utilizar transfer learning, se generó un modelo personalizado con una arquitectura similar a la utilizada anteriormente. En este caso, se reemplazó el modelo pre-entrenado de MobileNet por dos bloques de convolución, cada uno compuesto por combinaciones de dos secuencias de Conv2D, BatchNormalization, MaxPooling2D y una capa ReLU, como se muestra en [Figura 7a](#). Estas combinaciones de *Conv2D*, *Batch normalization* y *ReLU* fueron un factor común que vimos en distintos modelos para la competencia en [\[Gre19\]](#) y [\[BOU19\]](#), los cuales tomamos como referencia.

La arquitectura general del bloque de convolución se presenta en [Figura 7b](#). La razón de utilizar la capa de batch normalization posterior a cada capa de convolución y previo a *ReLU* es para normalizar la capa de activación lo cual hace que el modelo sea menos sensible a los pesos de inicialización.



(a) Diagrama de la arquitectura del bloque CovBlock



(b) Diagrama de la arquitectura del modelo custom made

En este modelo se conservan las características del modelo baseline planteadas anteriormente. Los valores se normalizan al rango $[-1;1]$ y se entrena utilizando el mismo conjunto de datos aumentado que para el modelo de transfer learning. El modelo obtuvo un lwlap de 0.28 en validación, ampliamente inferior al modelo basado en MobileNet. Durante el estudio de aumento de datos se utilizó una técnica que mejoro mucho el rendimiento, cuyo resultado se encuentra en la [Figura 11](#).

5. Mejoras y Optimización

5.1. Aumento de datos

Dado que los modelos presentan sobreajuste incluso después de implementar técnicas de regularización, se estima que el conjunto de entrenamiento curado no es lo suficientemente grande. En este contexto se aplicarán técnicas de aumento de datos para mejorar la generalización de los modelos, principalmente experimentando con el modelo de *transfer learning* basado en *MobileNet*.

5.1.1. Datos Ruidosos

Para la integración de los datos ruidosos en el modelo con transfer learning, inicialmente se entrenó con un modelo base utilizando los datos curados. Posteriormente, se añadieron progresivamente datos ruidosos al conjunto de entrenamiento, etiquetándolos con el modelo entrenado, y volviendo a entrenar el modelo con el nuevo conjunto aumentado. Se fue almacenando el mejor modelo obtenido a medida que avanzaba el entrenamiento. Este método fue inspirado en los trabajos de [\[Gre19\]](#) y [\[BOU19\]](#).

Este enfoque no fue suficiente como para mejorar el rendimiento de los modelos en validación, no hubieron cambios con diferencias apreciables frente a lo presentado en la [Figura 6](#). Se decidió continuar investigando otras técnicas de aumento de datos.

5.1.2. Transformaciones en audios

Como primer aproximación al aumento de datos se decide duplicar el conjunto de entrenamiento aplicando transformaciones a los audios, previo a la generación de espectrogramas. En particular se duplica el conjunto aplicando ruido Gaussiano, pitch shift, time shift y estiramiento de tiempo, donde cada transformación tiene una probabilidad del 50 % de aplicarse en el audio. Se aplicó la transformación sobre el conjunto de datos de entrenamiento y los resultados se muestran en la [Figura 6](#). Se observa un leve efecto regularizador en validación que hace que el rendimiento converja más rápido. Consideramos este resultado como bueno y por lo tanto se utiliza esta técnica por defecto en los experimentos futuros.

5.1.3. SpecAugment

[\[PCZ+19\]](#) define una técnica de aumento de datos específica para espectrogramas *log-mel* llamada *SpecAugment*, que consiste en la aplicación de varias transformaciones a las imágenes, tales como enmascarar bandas temporales y de frecuencia, o desplazar el audio en el tiempo. Se utilizó la implementación de [\[Ped21\]](#), cuyo código define una capa de *Keras* que ejecuta el algoritmo de aumento durante el entrenamiento. Se realizó un experimento utilizando los parámetros por defecto de la capa, y los resultados pueden observarse en la [Figura 8](#). En esta ocasión se incrementó el número de épocas a 20, se puede notar que el modelo subajusta a los datos de entrenamiento. Posteriormente, se realizó una búsqueda de los mejores parámetros para *SpecAugment* utilizando *Hyperband*, la cual concluyó en que el mejor resultado es no utilizar la capa (anulando todos los parámetros). De una breve investigación se encontró que la capa aplica aumento a la totalidad de los datos que ingresan al modelo, efectivamente disminuyendo la cantidad de

información del conjunto de entrenamiento, por lo tanto se creó una capa wrapper sobre la capa original que ejecuta el aumento con una probabilidad del 50 % para no deshacernos de los datos originales, efectivamente simulando la duplicación de los datos de entrenamiento al aplicar el aumento. Los resultados se encuentran en la misma gráfica y se observa que esta técnica logra reducir en gran medida el sobreajuste del modelo. Se hizo un experimento agregando *SpecAugment* al modelo custom y aumentando la cantidad de épocas de entrenamiento, cuyos resultados estan en la [Figura 11](#), mejorando mucho su rendimiento (lwlap 0.59), aun así se priorizó el modelo de *transfer learning* debido a que su entrenamiento es más rápido por lo que permite iterar más rápido.

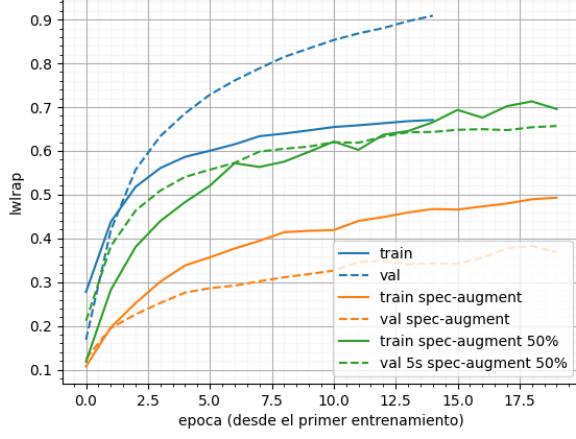


Figura 8: LwLrap de entrenamiento para modelo basado en MobileNet regularizado, utilizando SpecAugment y luego reduciendo su aplicación al 50 % de las muestras.

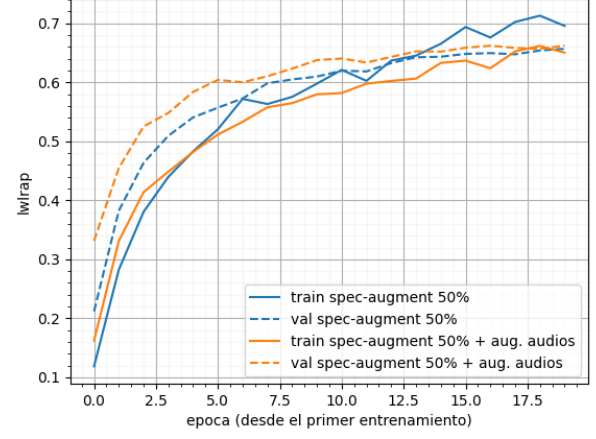


Figura 9: LwLrap de entrenamiento para modelo basado en MobileNet regularizado, utilizando SpecAugment y comparando con aumento de audios.

Se probó esta técnica para el conjunto de audios aumentados mediante transformaciones, cuyos resultados se comparan contra los anteriores en la [Figura 9](#). Utilizar *SpecAugment* en el conjunto aumentado logra eliminar completamente el sobreajuste y una mejora en el rendimiento en validación que sin embargo converge en las últimas épocas. Debido al poco sobreajuste se decide fine-tunear este modelo durante 25 épocas para hallar su límite, probando también disminuyendo la probabilidad de *SpecAugment* a 35 %. Se observa que este último logra superar al resto de técnicas probadas, con un *LWLRAP* de 0.69 en la [Figura 10](#).

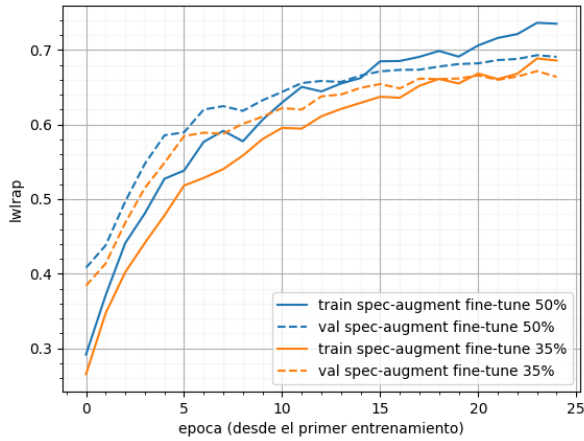


Figura 10: LwLrap de entrenamiento fine-tune para modelo basado en MobileNet regularizado utilizando aumento de audios y SpecAugment al 50 % y 35 %.

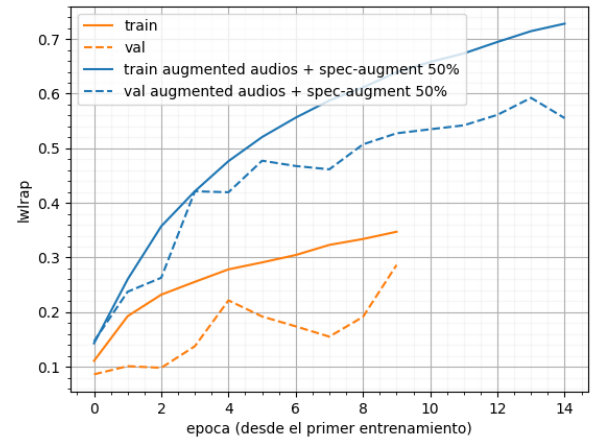


Figura 11: LwLrap de entrenamiento para modelo custom made, sin aumento y utilizando SpecAugment al 50 %.

5.1.4. MixUp

El siguiente enfoque utilizado fue la técnica de *MixUp*, una estrategia de aumento de datos que implica mezclar pares de ejemplos de audio para generar datos de entrenamiento aumentados. Este método combina tanto los ejemplos de audio como las etiquetas. Este enfoque puede resultar beneficioso, ya que obliga al modelo a deducir información útil en contextos que podrían considerarse ruidosos por la mezcla de sonidos.

Para generar las nuevas instancias de entrenamiento, el proceso comienza seleccionando pares de audios. Inicialmente, se verifica y se toma la longitud mínima entre los dos audios para asegurarnos de que la mezcla se realice en las secciones donde ambos coinciden. Si uno de los audios es más largo que otro, el segmento sobrante se añade al final del audio mezclado para conservar toda la información posible. Las etiquetas de ambos audios originales se combinan, asignándose al nuevo audio mezclado.

Esta técnica disminuye el sobreajuste marcado en algunos de los modelos donde la cantidad de muestras no es suficiente para su complejidad, aunque no lo controla tan bien como *SpecAugment*. En particular se pueden observar los resultados en [este experimento de Comet](#). Se probó la técnica en combinación a *SpecAugment* ([experimento de Comet](#)) pero los resultados obtenidos no fueron buenos en este caso, pues el modelo vuelve a presentar sobreajuste y esta vez con menor rendimiento en validación. Debido a la menor efectividad y la mayor complejidad en implementación decidimos quedarnos solamente *SpecAugment*.

6. Análisis de resultados

6.1. Registro de experimentos

Se utilizó Comet para registrar y monitorear todos los experimentos realizados durante el desarrollo del proyecto. La utilización de *Comet* nos permitió monitorear en tiempo real, observando las métricas en el tablero de *Comet* y comparar los experimentos ayudándonos a identificar las mejores estrategias y configuraciones. Además, se utilizó para generar gráficas por experimento sin tener que generarlas con datos locales.

Por un error inicial los experimentos fueron almacenados en dos proyectos, la mayoría de ellos se encuentran en [Experimentos Comet 1](#)¹, y otra pequeña sección de las pruebas fueron realizadas en [Experimentos Comet 2](#).

6.2. Resultados de Kaggle

Los experimentos que tuvieron mejores métricas en validación son los que fueron probados en el conjunto de prueba a partir de *Kaggle*. El modelo generado a partir de *transfer learning*, con un aumento de datos a partir de transformaciones en el audio, en validación obtiene un *LWLRAP* de 0.66, cuyos resultados se pueden visualizar en el [Experimento 1 \(Comet ML\)](#), este cual obtuvo en test un *LWLRAP* de 0.48. Sin embargo, el modelo con el que se obtuvo el mejor resultado en validación no fue ese, sino que el modelo que utiliza además *SpecAugment* a 35 %, cuyos resultados están disponibles en el [Experimento 2 \(Comet ML\)](#), sin embargo no se pudo enviar una submission de este modelo debido al límite de diario de submissions impuesto en el challenge. Este último modelo se selecciona como el mejor encontrado.

7. Conclusiones

En este trabajo se desarrolló y evaluó una serie de modelos de aprendizaje automático para el etiquetado de audio utilizando técnicas como transfer learning y redes neuronales convolucionales. Destacamos los siguientes aspectos:

- La efectividad del *transfer learning*, en particular utilizando *MobileNetV2*, para aprovechar conocimientos previos y mejorar significativamente la precisión en la clasificación de audio.
- La utilidad de técnicas de procesamiento y aumento de datos como *MixUp* y *SpecAugment*, que, eventualmente contribuyeron a mejorar la generalización del modelo. Se redujo de manera significativa la diferencia entre el rendimiento del conjunto de entrenamiento y el de validación.
- La utilización de *Comet* como herramienta para el seguimiento y registro de los experimentos, facilitando la comparación entre diferentes configuraciones y la reproducción de resultados.

Los resultados obtenidos indican que mientras los modelos basados en *transfer learning* mostraron un rendimiento superior, la integración de técnicas de aumento de datos y aplicadas podría mejorar aún más los resultados, particularmente en conjuntos de datos desbalanceados o en condiciones de baja señal a ruido.

7.1. Limitaciones y Trabajo Futuro

A pesar de los resultados, esta tarea tiene varias limitaciones que podrían tratarse como mejoras. Entre ellas, la necesidad de explorar configuraciones más complejas de los modelos y las técnicas de aumento utilizadas. Particularmente se puede investigar mucho más a fondo el uso de los audios del conjunto ruidoso que no fue aprovechado lo suficiente en este proyecto.

¹Dentro del workspace lab2 se encuentran separados en dos proyectos uno correspondiente al modelo Custom made y otro a MobileNet llamados Baseline-hmade y Baseline-mobilenet respectivamente

Referencias

- [BOU19] Eric BOUTEILLON. freesound-audio-tagging-2019. <https://github.com/ebouteillon/freesound-audio-tagging-2019>, 2019.
- [Gre19] Simon Grest. Kaggle freesound audio tagging competition. <https://github.com/simongrest/kaggle-freesound-audio-tagging-2019?tab=readme-ov-file>, 2019.
- [PCZ⁺19] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. Specaugment: A simple data augmentation method for automatic speech recognition. In *Interspeech 2019*, interspeech2019.ISCA, September 2019.
- Ped21] Miguel Otero Pedrido. spec_augment. https://github.com/MichaelisTrofficus/spec_augment, 2021.